# Arquitectura de computadoras

Sistemas Operativos · Semana 1

Cap. 1: Introducción (temas 1.1 – 1.3)

Rubén Rumipamba

Escuela Politécnica Nacional

# Agenda de la semana

**SESIÓN 1 · 2 H**

## 1.1 Bloques fundamentales

- Von Neumann y Harvard
- CPU, memoria, E/S y buses
- De la placa real al modelo
- Demo Linux / Windows · quiz

**SESIÓN 2 · 2 H**

## 1.2 Funcionamiento

- Ciclo de instrucción
- Traza en la máquina de Stallings
- Interrupciones
- E/S programada, por interrupciones y DMA

**SESIÓN 3 · 1 H**

## 1.3 Métricas + consolidación

- Latencia, throughput, tiempo de acceso
- T = IC × CPI × Tc · Amdahl
- Taller de ejercicios · quiz
- Deber 1 (en parejas)

# Objetivos de la semana

| | | |
|---|---|---|
| 1 | Identificar los bloques funcionales de una computadora y verificarlos en un equipo real con comandos de Linux y Windows. | Sesión 1<br>RA1 |
| 2 | Explicar el ciclo de instrucción, las interrupciones y las técnicas de E/S como base del funcionamiento del SO. | Sesión 2<br>RA1 |
| 3 | Calcular e interpretar métricas de rendimiento y automatizar un primer inventario de hardware con scripts. | Sesión 3<br>RA1 · RA2 |

# Sesión 1 · Bloques fundamentales de una computadora

Tema 1.1 · 2 horas

# ¿Por qué empezar por el hardware?




*Visión en capas del sistema de cómputo*

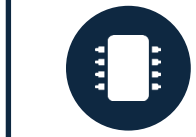
**El SO administra...**

**Tiempo de CPU**
¿Qué programa se ejecuta ahora y durante cuánto tiempo?

**Espacio de memoria**
¿Dónde vive cada programa y quién puede leer qué?

**Acceso a E/S**
¿Quién usa el disco, la red o la impresora, y en qué orden?

# Arquitectura de Von Neumann



## CPU

- Unidad de control
- ALU
- Registros



## Memoria principal

- Instrucciones
- Datos
- (mismo espacio)



## Módulos de E/S

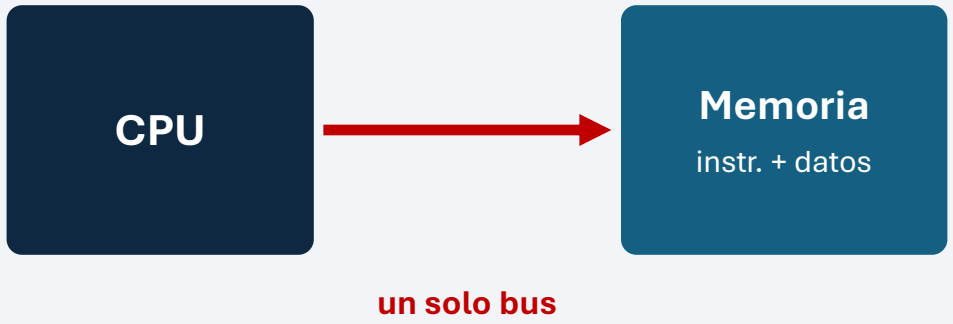
- Teclado, disco, red…
- Controladores
- Registros de E/S

**Bus del sistema (datos · direcciones · control)**

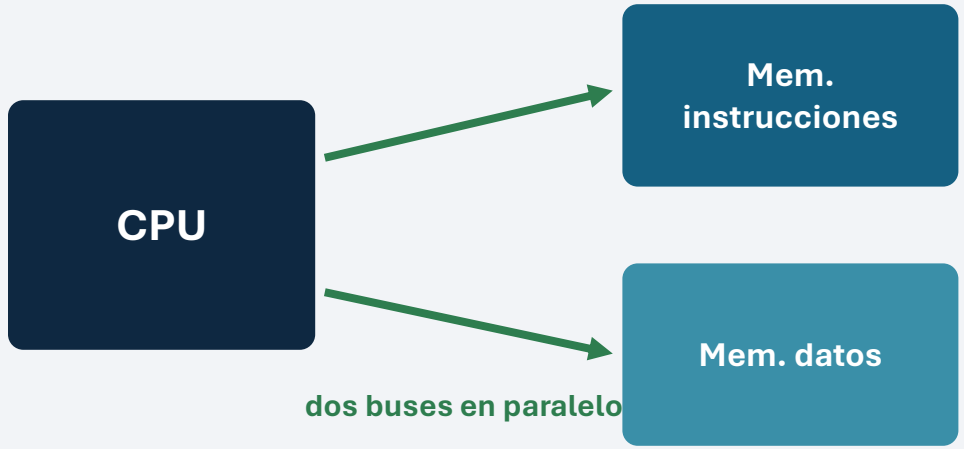
**Programa almacenado:** instrucciones y datos comparten la misma memoria, en binario, y se acceden por dirección.

# Von Neumann frente a Harvard

## Von Neumann



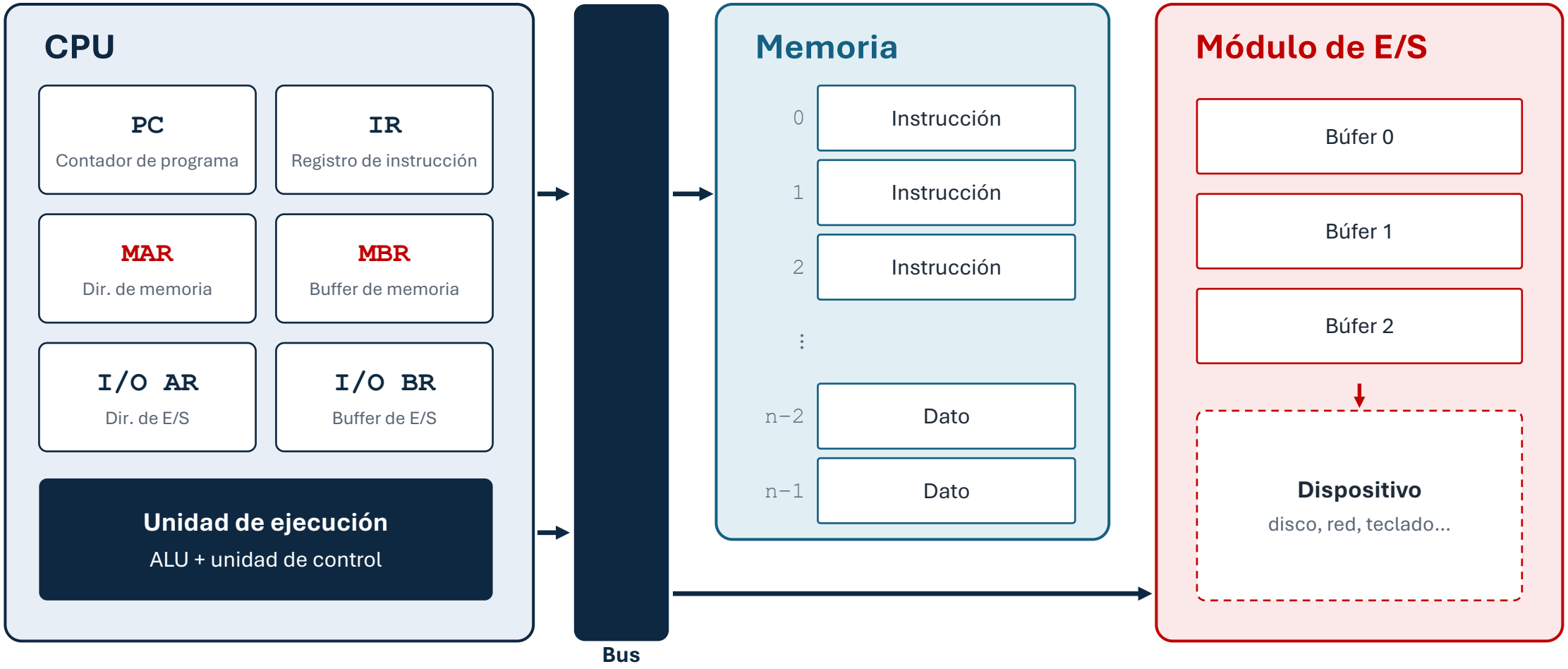

## Harvard




### ¿Dónde se usa hoy?

- CPU modernas: cachés L1i y L1d separadas (Harvard modificada)
- Microcontroladores: AVR (Arduino), PIC
- Procesadores de señales (DSP)

# Vista de alto nivel de una computadora



# La CPU por dentro: control, ALU y registros

### Unidad de control

Decodifica instrucciones y genera las señales de control al ritmo del reloj.

### ALU

Operaciones aritméticas y lógicas; actualiza los indicadores (flags).

### Registros

Almacenamiento más rápido, dentro de la CPU. Pocos y pequeños.

| Registro | Función |
|---|---|
| `PC` | Dirección de la próxima instrucción |
| `IR` | Instrucción en ejecución |
| `MAR` | Dirección de memoria a leer o escribir |
| `MBR` | Dato leído o por escribir en memoria |
| `AC` | Acumulador: resultado de la ALU |
| `PSW` | Estado: flags y modo usuario/núcleo |
| `SP` | Puntero de pila (llamadas, contexto) |

# Memoria principal

*contenido (1 byte por celda)*

| 1A | 00 | FF | 2F | 7C | 03 | E1 | 90 |
|---|---|---|---|---|---|---|---|
| 0x0000 | 0x0001 | 0x0002 | **0x0003** | 0x0004 | 0x0005 | 0x0006 | 0x0007 |

*dirección*

### Volátil

La RAM pierde su contenido al apagar. El SO se carga desde disco en cada arranque.

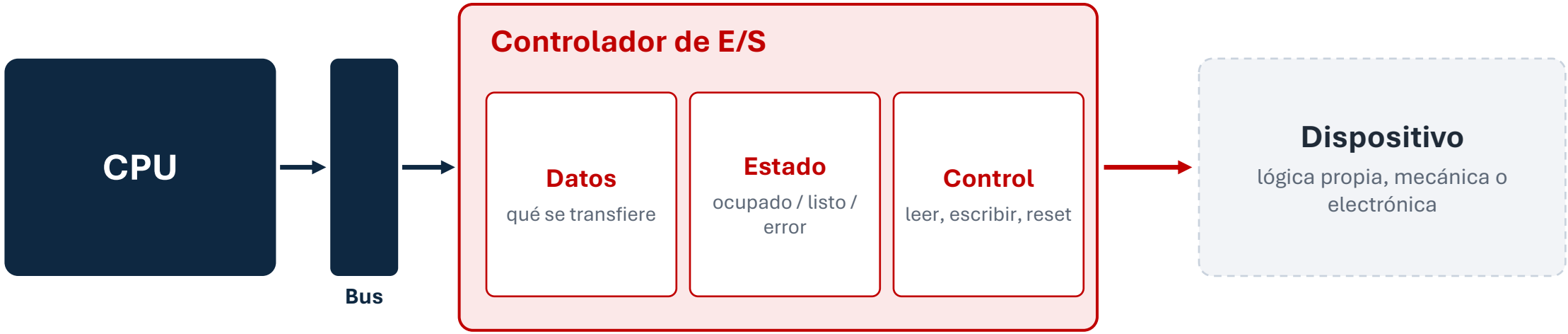
### Acceso aleatorio

Cualquier dirección se lee en el mismo tiempo, sin recorrer las anteriores.

### RAM y ROM/Flash

El firmware (BIOS/UEFI) vive en memoria no volátil; programas y datos, en RAM.

# Módulos de E/S: el puente con los dispositivos



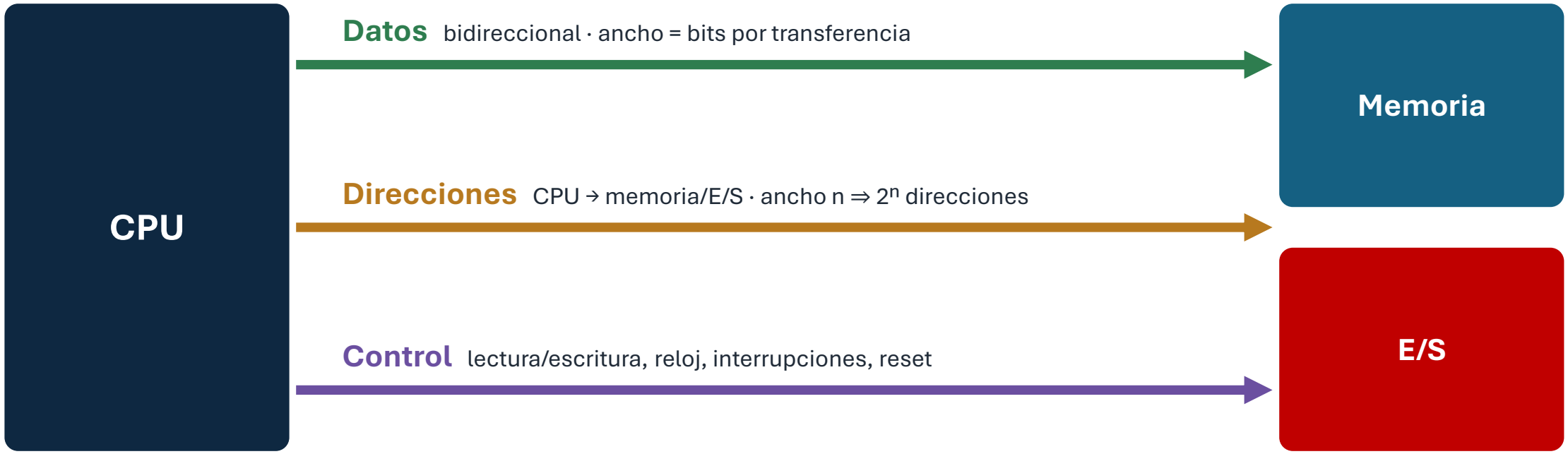

**Entrada**
teclado, ratón, sensores

**Salida**
pantalla, impresora

**Almacenamiento**
SSD, HDD, USB

**Comunicación**
Ethernet, Wi-Fi

# Buses del sistema: datos, direcciones y control




**Ejemplo:** bus de direcciones de 32 bits → $2^{32}$ direcciones de byte = 4 GiB de memoria direccionable.

# Ejercicio rápido · Buses

Un procesador tiene un **bus de direcciones de 36 bits** y un **bus de datos de 64 bits**. Cada dirección apunta a un byte.

1. ¿Cuánta memoria puede direccionar como máximo? Exprésalo en GiB.
2. ¿Cuántos bytes se transfieren en cada ciclo de bus?
3. Si el bus trabaja a 100 millones de transferencias por segundo, ¿cuál es su ancho de banda máximo en MB/s?

# Ejercicio rápido · Solución

1

**Memoria máxima**

$2^{36} = 2^{6} \times 2^{30}$ B

**= 64 GiB**

2

**Por transferencia**

64 bits ÷ 8

**= 8 bytes**

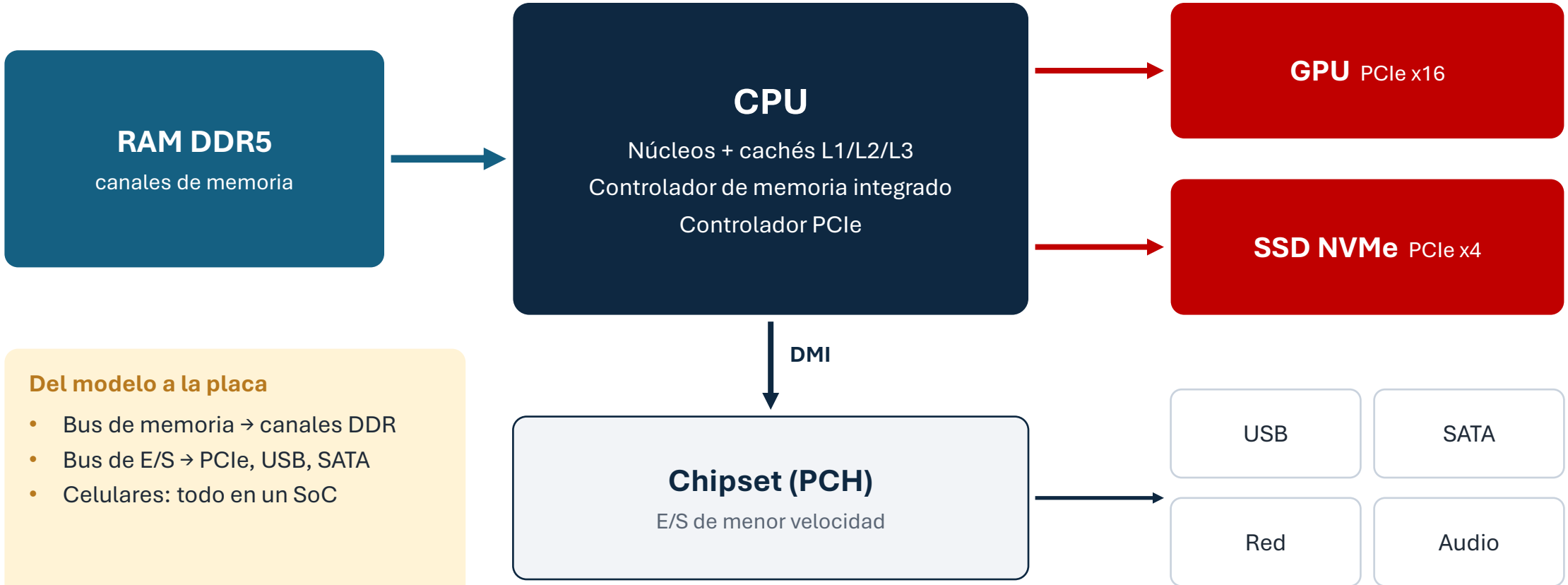
3

**Ancho de banda**

8 B × 100 × $10^{6}$ /s

**= 800 MB/s**

# De la placa real al modelo




**Del modelo a la placa**

- Bus de memoria → canales DDR
- Bus de E/S → PCIe, USB, SATA
- Celulares: todo en un SoC

# Demo: veamos los bloques en un equipo real

| Bloque | Linux (Bash) | Windows (PowerShell) |
|---|---|---|
| **CPU** | `lscpu` · `cat /proc/cpuinfo` | `Get-CimInstance Win32_Processor` |
| **Memoria** | `free -h` · `cat /proc/meminfo` | `Get-CimInstance Win32_PhysicalMemory` |
| **Almacenamiento** | `lsblk` · `df -h` | `Get-PhysicalDisk` · `Get-Volume` |
| **Buses y E/S** | `lspci` · `lsusb` | `Get-PnpDevice -PresentOnly` |
| **Resumen** | `sudo lshw -short` | `systeminfo` · `msinfo32` |

**Reto en parejas:** ejecuten las filas CPU y Memoria en sus equipos y comparen núcleos, hilos y RAM.

# ¿Qué nos dice la salida?

bash · salida de ejemplo

```
$ lscpu
Architecture:            x86_64
CPU(s):                  8
Model name:              Intel(R) Core(TM) i7-1165G7
Thread(s) per core:      2
Core(s) per socket:      4
L1d cache:               192 KiB (4 instances)
L1i cache:               128 KiB (4 instances)
L2 cache:                5 MiB (4 instances)
L3 cache:                12 MiB (1 instance)
```

```
PS> Get-CimInstance Win32_Processor |
      Format-List NumberOf*, MaxClockSpeed
NumberOfCores : 4   NumberOfLogicalProcessors : 8
MaxClockSpeed : 2803
```

**L1d y L1i separadas**
Harvard modificada en la caché L1

**2 hilos × 4 núcleos = 8 CPU lógicas**
Multithreading por hardware (Cap. 4)

**L2 y L3**
Jerarquía de memoria (semana 2)

**2803 MHz ≈ 2,8 GHz**
Frecuencia de reloj (sesión 3)

# Quiz Sesión 1 · Pregunta 1 de 2



**En la arquitectura de Von Neumann, las instrucciones y los datos…**

**A)** se guardan en memorias separadas

**B)** comparten la misma memoria y el mismo bus

**C)** solo los datos se guardan en memoria

**D)** se guardan siempre en ROM

# Quiz Sesión 1 · Respuesta 1 de 2

**Pregunta:** En la arquitectura de Von Neumann, las instrucciones y los datos…

A) se guardan en memorias separadas

**B) comparten la misma memoria y el mismo bus** ✓

C) solo los datos se guardan en memoria

D) se guardan siempre en ROM

### ¿Por qué?

Es la idea de **programa almacenado**: instrucciones y datos viven en la misma memoria y viajan por el mismo bus.

La opción A describe la arquitectura **Harvard**.

Consecuencia: el **cuello de botella de Von Neumann**, que las cachés L1i/L1d separadas ayudan a aliviar.

# Quiz Sesión 1 · Pregunta 2 de 2

**¿Qué registro de la CPU guarda la dirección de memoria que se va a leer o escribir?**

**A)** PC

**B)** IR

**C)** MAR

**D)** AC

# Quiz Sesión 1 · Respuesta 2 de 2

**Pregunta:** ¿Qué registro de la CPU guarda la dirección de memoria que se va a leer o escribir?

A) PC

B) IR

C) **MAR** ✓

D) AC

**¿Por qué?**

**MAR** (Memory Address Register) contiene la dirección que la CPU pone en el bus de direcciones.

**PC** guarda la dirección de la PRÓXIMA instrucción; en la búsqueda se copia al MAR.

**IR** guarda la instrucción en ejecución y **AC** el resultado de la ALU.

# Cierre de la sesión 1

### Programa almacenado

Instrucciones y datos en la misma memoria (Von Neumann). Las cachés L1 separadas hacen a las CPU actuales «Harvard modificada».

### Cuatro bloques

CPU, memoria, módulos de E/S y buses. MAR y MBR son la puerta de la CPU hacia la memoria.

### Todo es verificable

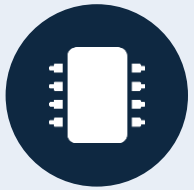
lscpu, free, lsblk, lspci y sus equivalentes en PowerShell muestran el modelo en un equipo real.

**Próxima sesión:** ciclo de instrucción, interrupciones y técnicas de E/S. **Tarea:** formar parejas y crear el repositorio del portafolio.

# Sesión 2 · Funcionamiento de la computadora

Tema 1.2 · 2 horas

# Repaso rápido de la sesión 1



**Programa almacenado significa...**



**El PC contiene...**

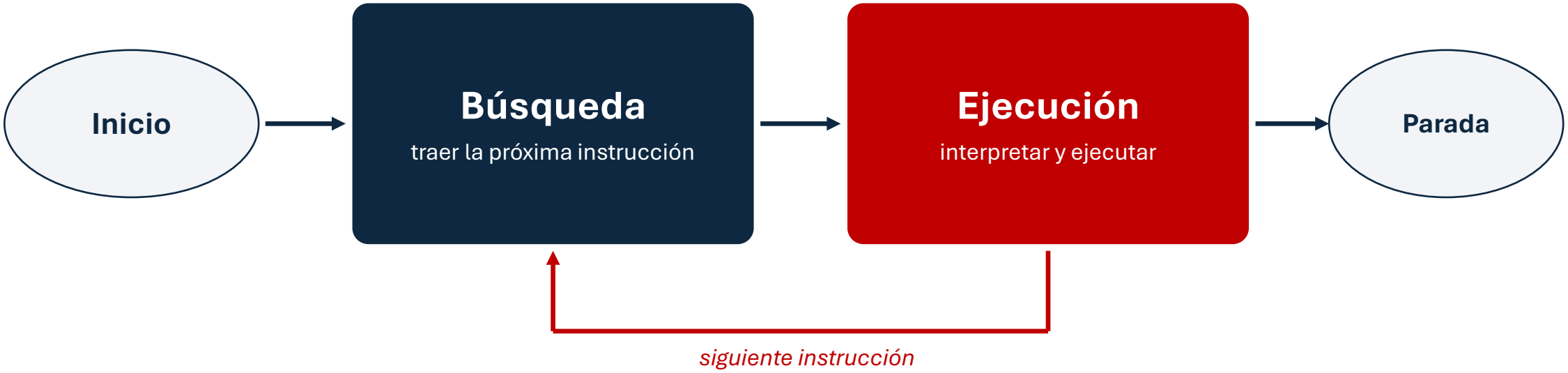


**MAR y MBR sirven para...**



**Con n bits de direcciones se direccionan...**

# El ciclo de instrucción




**Ciclo de instrucción** = ciclo de búsqueda + ciclo de ejecución. Se repite millones de veces por segundo, siempre guiado por el **PC**.

# La búsqueda (fetch), paso a paso

**1**

**`MAR ← PC`**

La dirección de la instrucción sale al bus de direcciones.

**2**

**`MBR ← M[MAR]`**

La memoria devuelve la palabra por el bus de datos.

**3**

**`IR ← MBR`**

La instrucción queda en el registro de instrucción.

**4**

**`PC ← PC + 1`**

El PC ya apunta a la siguiente instrucción.

**Notación RTL:** **`A ← B`** se lee «A recibe el contenido de B». **`M[X]`** es la palabra de memoria en la dirección X.

Laboratorio de la CPU · Paso

# La ejecución: cuatro tipos de acciones



### Procesador ↔ memoria

Transferir datos entre CPU y memoria.

**Ej.:** `LOAD, STORE`



### Procesador ↔ E/S

Enviar o recibir datos de un módulo de E/S.

**Ej.:** `IN, OUT`



### Procesamiento de datos

Operaciones aritméticas o lógicas en la ALU.

**Ej.:** `ADD, SUB, AND, CMP`



### Control

Alterar la secuencia: cambiar el valor del PC.

**Ej.:** `JMP, CALL, RET`

# La máquina hipotética de Stallings

### Formato de instrucción (16 bits)

| Opcode | Dirección |
|---|---|
| bits 0–3 | bits 4–15 · 4096 palabras |

### Formato de entero (16 bits)

| S | Magnitud |
|---|---|
| | bits 1–15 |

### Registros

| PC | IR | AC |
|---|---|---|
| próxima instrucción | instrucción actual | acumulador |

| Opcode | Instr. | Efecto |
|---|---|---|
| 0001 = 1 | LOAD X | AC ← M[X] |
| 0010 = 2 | STORE X | M[X] ← AC |
| 0101 = 5 | ADD X | AC ← AC + M[X] |

**Lectura en hexadecimal**

**1940** → opcode **1** (LOAD), dirección **940**

Cada dígito hex = 4 bits

# Ejercicio: traza de un programa

| Dir. | Contenido | Significado |
|---|---|---|
| 300 | **1940** | LOAD 940 |
| 301 | **5941** | ADD 941 |
| 302 | **2941** | STORE 941 |
| 940 | **0003** | dato = 3 |
| 941 | **0002** | dato = 2 |

**Estado inicial:** `PC = 300`, AC = 0000

| Al terminar | PC | IR | AC | M[941] |
|---|---|---|---|---|
| Instrucción 1 | ? | ? | ? | ? |
| Instrucción 2 | ? | ? | ? | ? |
| Instrucción 3 | ? | ? | ? | ? |

**Recuerda:** en cada instrucción primero ocurre la búsqueda (IR ← instrucción, PC ← PC + 1) y luego la ejecución.

# Traza · Solución

| Al terminar | PC | IR | AC | M[941] |
|---|---|---|---|---|
| 1 · LOAD 940 | 301 | 1940 | 0003 | 0002 |
| 2 · ADD 941 | 302 | 5941 | 0005 | 0002 |
| 3 · STORE 941 | 303 | 2941 | 0005 | **0005** |

**Observa:** el PC ya vale 303 al terminar la tercera instrucción, aunque 303 aún no se ha ejecutado. El PC siempre va «un paso adelante».

### ¿Qué hizo el programa?

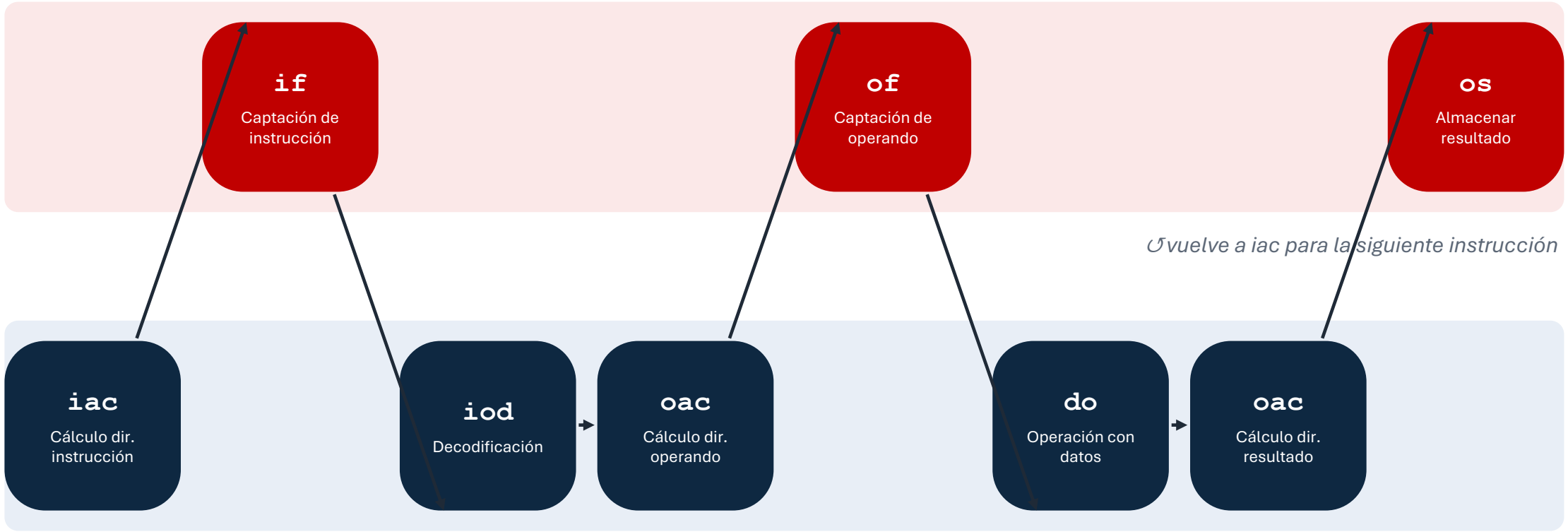
Sumó `M[940] + M[941]` = 3 + 2 y guardó el resultado **5** en la dirección 941.

Cada instrucción: 1 búsqueda + 1 ejecución.

Laboratorio de la CPU · pestaña 1

# Diagrama de estados del ciclo de instrucción

**Accesos a memoria o E/S**




**Operaciones internas de la CPU**

# Interrupciones: ¿qué son y de dónde vienen?






**De programa**

División por cero, desbordamiento, acceso ilegal a memoria.



**De temporizador**

El reloj interrumpe periódicamente: base de la multitarea.



**De E/S**

Un controlador avisa que terminó una operación o que hubo un error.



**Fallo de hardware**

Error de paridad de memoria, caída de energía.

**Idea clave:** la CPU no espera a los dispositivos lentos; sigue trabajando y el dispositivo le avisa cuando termina.

# Sin interrupciones frente a con interrupciones

## Sin interrupciones

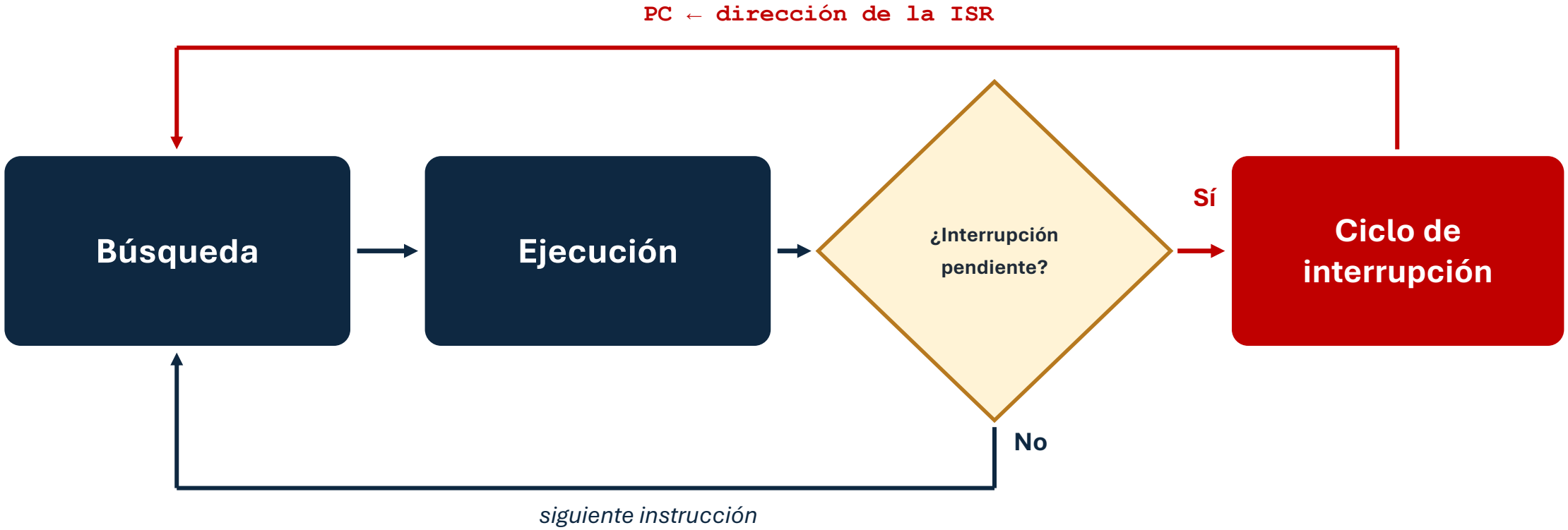



## Con interrupciones




**Ganancia:** la CPU y el dispositivo trabajan en paralelo. La CPU solo se detiene un momento para la ISR.

# El ciclo de instrucción con interrupciones




**La comprobación ocurre al terminar cada instrucción**: nunca a la mitad.

# ¿Qué ocurre en el ciclo de interrupción?

1. **Termina la instrucción actual**
2. **Guarda el contexto (PC, PSW)**
3. **PC ← dirección de la ISR**
4. **Ejecuta la ISR (código del SO)**
5. **Restaura el contexto y continúa**

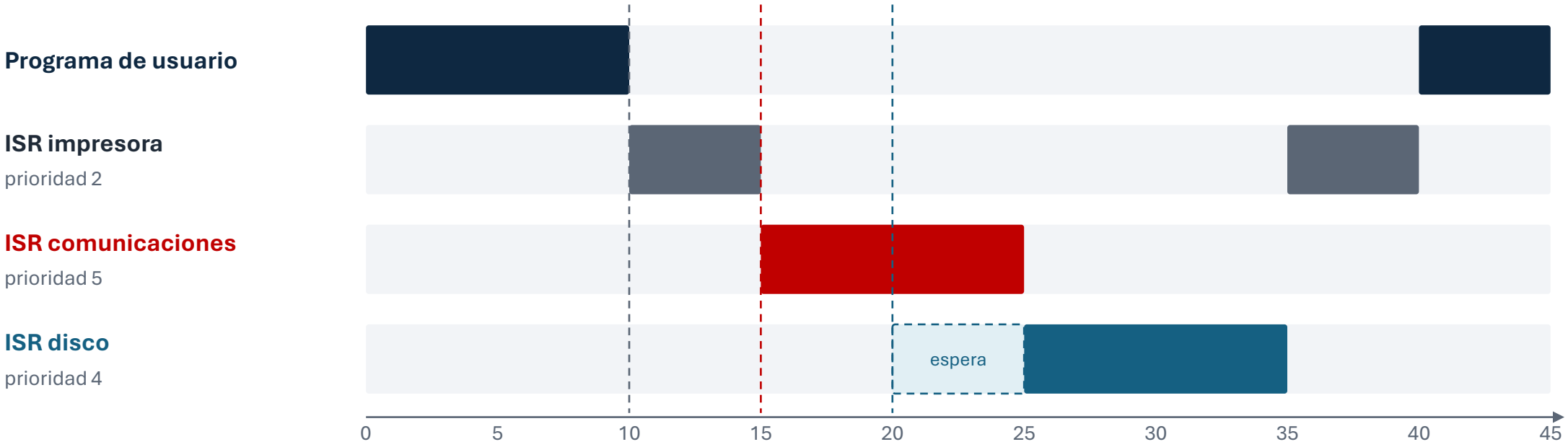
**Conexión con el SO**

Guardar y restaurar el contexto es la misma idea que usará el SO para **cambiar de un proceso a otro** (Cap. 4.5).

**Laboratorio de la CPU** · pestaña 1

La ISR del ejemplo cuenta interrupciones en la dirección 942 y vuelve con IRET.

# Interrupciones múltiples




## Secuencial

Se deshabilitan las interrupciones durante la ISR. Simple, pero ignora la urgencia.

## Anidada por prioridad

Una interrupción más prioritaria interrumpe a la ISR en curso (ejemplo de arriba).

# Tres técnicas de E/S

| | E/S programada | E/S por interrupciones | DMA |
|---|---|---|---|
| **¿Quién mueve los datos?** | CPU | CPU (palabra a palabra) | **Controlador DMA** |
| **¿La CPU espera?** | **Sí: sondeo activo (polling)** | No: sigue con otro trabajo | No: solo inicia y recibe aviso |
| **Interrupciones** | Ninguna | Una por palabra o bloque pequeño | Una al final del bloque |
| **Uso típico** | Dispositivos muy simples, arranque | Teclado, ratón, puertos serie | Discos, SSD, red, GPU |

**De izquierda a derecha:** menos trabajo para la CPU y más hardware especializado.

DMA: Acceso Directo a Memoria

# Acceso directo a memoria (DMA)

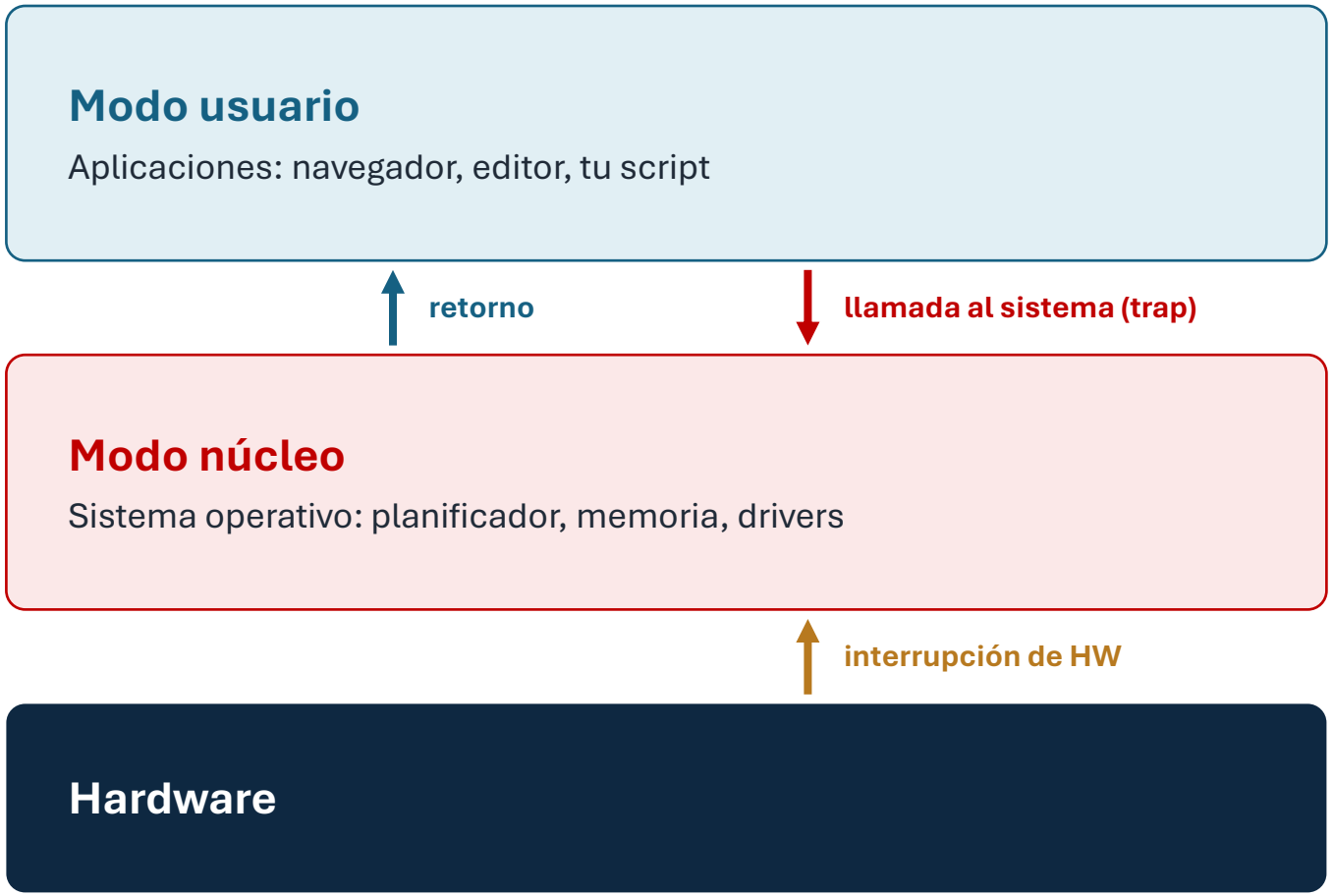



**1 La CPU programa el DMA**
dirección de memoria, cantidad de palabras, lectura o escritura.

**2 Transferencia directa**
dispositivo ↔ memoria, sin pasar por la CPU (robo de ciclos de bus).

**3 Una interrupción al final**
la CPU solo interviene al inicio y al final del bloque.

# Puente al SO: modo usuario y modo núcleo




## Ideas clave

- Un bit del **PSW** indica el modo actual.
- Las instrucciones privilegiadas solo se ejecutan en modo núcleo.
- Se entra al núcleo solo por **traps, interrupciones o excepciones**.
- El SO es un programa **dirigido por eventos**.

# Quiz Sesión 2 · Pregunta 1 de 2

**¿En qué momento comprueba la CPU si hay interrupciones pendientes?**

**A)** Al inicio de la búsqueda de cada instrucción

**B)** A la mitad de la ejecución

**C)** Al terminar la ejecución de cada instrucción

**D)** Solo cuando el programa lo solicita

# Quiz Sesión 2 · Respuesta 1 de 2

**Pregunta:** ¿En qué momento comprueba la CPU si hay interrupciones pendientes?

**A)** Al inicio de la búsqueda de cada instrucción

**B)** A la mitad de la ejecución

**C)** **Al terminar la ejecución de cada instrucción** ✓

**D)** Solo cuando el programa lo solicita

### ¿Por qué?

La comprobación ocurre **al terminar el ciclo de ejecución**, antes de la siguiente búsqueda.

Así la instrucción en curso termina completa y el estado (PC, registros) queda consistente para guardarlo.

La opción D describe una **llamada al sistema** (trap), que es otro mecanismo.

# Quiz Sesión 2 · Pregunta 2 de 2

**La CPU atiende la ISR de la impresora (prioridad 2) y llega una interrupción del disco (prioridad 4). Con interrupciones anidadas, ¿qué ocurre?**

**A)** Se ignora la del disco

**B)** El disco espera a que termine la impresora

**C)** Se suspende la ISR de la impresora y se atiende el disco

**D)** Se reinicia la ISR de la impresora

# Quiz Sesión 2 · Respuesta 2 de 2

**Pregunta:** Impresora (prio. 2) en curso y llega el disco (prio. 4), con anidamiento...

**A)** Se ignora la del disco

**B)** El disco espera a que termine la impresora

**C) Se suspende la ISR de la impresora y se atiend el disco**

**D)** Se reinicia la ISR de la impresora

### ¿Por qué?

Con anidamiento, una interrupción de **mayor prioridad** interrumpe a la ISR en curso.

Se guarda el contexto de la ISR de la impresora, se ejecuta la del disco y luego se retoma la impresora.

La opción B sería el enfoque **secuencial**.

# Cierre de la sesión 2

### Búsqueda + ejecución

La CPU repite el ciclo guiada por el PC. En la búsqueda: MAR ← PC, MBR ← M[MAR], IR ← MBR, PC ← PC + 1.

### Interrupciones

Permiten solapar CPU y E/S. Se comprueban al final de cada instrucción; el contexto se guarda y se restaura.

### E/S eficiente

Programada → por interrupciones → DMA: cada paso libera más a la CPU.

**Próxima sesión (1 h):** métricas de rendimiento, taller de ejercicios y quiz. **Traer calculadora.**

# Sesión 3 · Métricas y consolidación

Tema 1.3 · Taller y quiz · 1 hora

# Latencia y throughput



## Latencia

Tiempo que tarda UNA operación, desde que se pide hasta que termina.

**Unidades:** ns · µs · ms

**Analogía:** *Tiempo que tarda un auto en cruzar el peaje.*



## Throughput

Cantidad de operaciones o datos completados por unidad de tiempo.

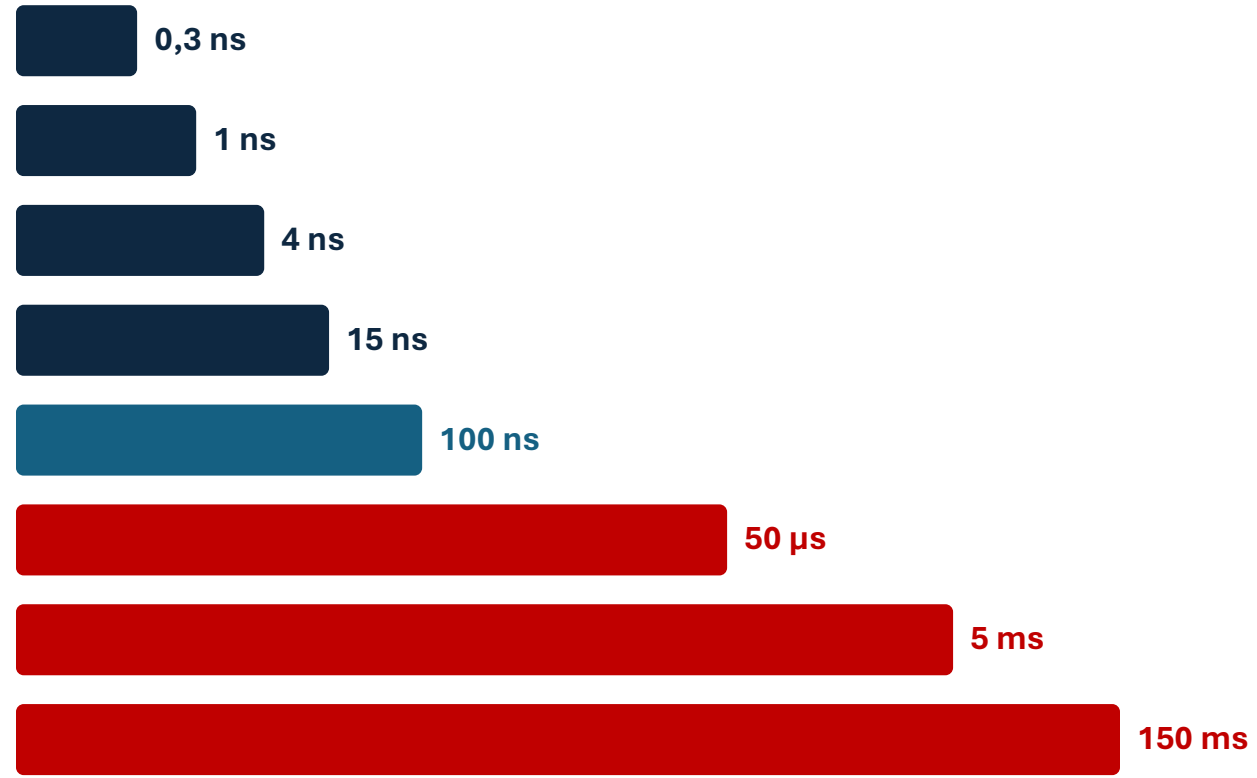
**Unidades:** operaciones/s · MB/s

**Analogía:** *Autos que cruzan el peaje por hora.*

**Ojo:** mejorar una no implica mejorar la otra. Más casetas suben el throughput, pero cada auto tarda lo mismo.

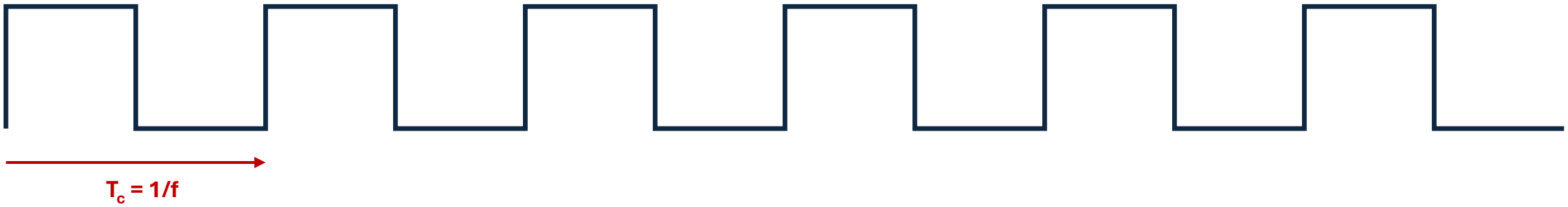
# Tiempo de acceso: de registros a la red

| Dispositivo | Tiempo de acceso (escala logarítmica) | Si 1 ns = 1 s |
|---|---|---|
| **Registro** | **0,3 ns** | 0,3 s |
| **Caché L1** | **1 ns** | 1 s |
| **Caché L2** | **4 ns** | 4 s |
| **Caché L3** | **15 ns** | 15 s |
| **RAM (DRAM)** | **100 ns** | 1,7 min |
| **SSD NVMe** | **50 µs** | 14 horas |
| **Disco duro (HDD)** | **5 ms** | 58 días |
| **Paquete Quito ↔ Madrid** | **150 ms** | 4,8 años |



*Valores aproximados; varían según el hardware y la red.*

# Reloj, CPI y MIPS




### Frecuencia y período

***f*** = ciclos por segundo (GHz)

$T_c = 1 / f$

3 GHz → $T_c \approx 0{,}33$ ns

### CPI

Ciclos por instrucción.

Promedio ponderado según la mezcla de instrucciones del programa.

### MIPS

$MIPS = f / (CPI \times 10^6)$

No compara bien arquitecturas con repertorios distintos.

# La ecuación del rendimiento de la CPU

$$T_{CPU} = IC \times CPI \times T_c = IC \times CPI / f$$

| Decisión de diseño | IC | CPI | Tc |
|---|---|---|---|
| Algoritmo y programa | ✓ | ✓ | |
| Compilador | ✓ | ✓ | |
| Repertorio de instrucciones (ISA) | ✓ | ✓ | ✓ |
| Organización (pipeline, cachés) | | ✓ | ✓ |
| Tecnología de fabricación | | | ✓ |

# Ley de Amdahl: el límite de mejorar una parte

**Speedup = 1 / [ (1 – f) + f / S ]**

f: fracción mejorable · S: factor de mejora

**Límite (S → ∞): Speedup → 1 / (1 – f)**

Laboratorio de la CPU · pestaña 3: Amdahl

**Ejemplo: f = 0,8 y S = 4**

Antes: 100 unidades de tiempo




Después: 20 + 80/4 = 40

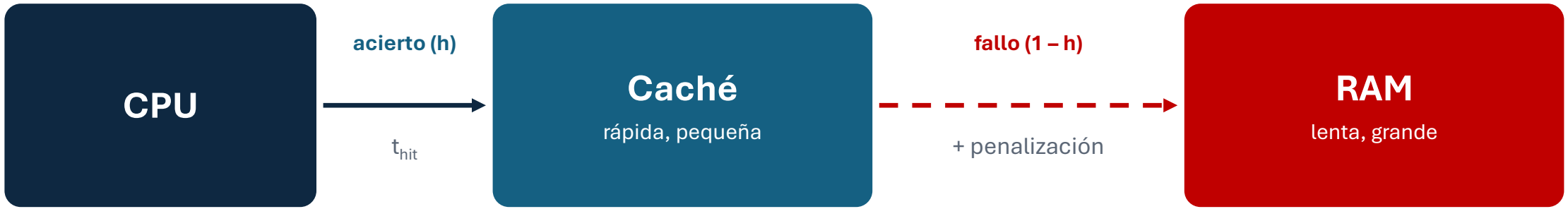



Speedup = 100 / 40 = **2,5×**

La parte no mejorada (20) ya es la mitad del tiempo.

# Tiempo medio de acceso a memoria (AMAT)

$$\text{AMAT} = t_{hit} + (1 - h) \times \text{penalización por fallo}$$




**Intuición:** con h alta, casi todos los accesos se resuelven en la caché y la memoria «parece» rápida. Más en la semana 2 (jerarquía de memoria).

# Taller · Ejercicio 1: ¿más GHz es más rápido?

**Ejercicio 1**

**Un programa ejecuta $10^9$ instrucciones con esta mezcla:**

| Tipo | Frecuencia | CPI (A) | CPI (B) |
|---|---|---|---|
| Aritmético-lógicas | 50 % | 1 | 1 |
| Carga/almacenamiento | 30 % | 2 | 3 |
| Saltos | 20 % | 3 | 4 |
| **Frecuencia de reloj** | | **3 GHz** | **4 GHz** |

**Calcula**

a) CPI promedio de A y de B

b) Tiempo de CPU de cada una

c) MIPS de cada una

d) ¿Cuál es más rápida y cuántas veces?

**Fórmulas:** $CPI = \Sigma\ CPI_i \times frecuencia_i$  $T = IC \times CPI / f$  $MIPS = f / (CPI \times 10^6)$

# Ejercicio 1 · Solución

## CPU A · 3 GHz

**CPI =** 0,5·1 + 0,3·2 + 0,2·3 = **1,7**

**T =** $10^9 \times 1{,}7 / (3 \times 10^9)$
**= 0,567 s**

**MIPS =** 3000 / 1,7 ≈ 1765

## CPU B · 4 GHz

**CPI =** 0,5·1 + 0,3·3 + 0,2·4 = **2,2**

**T =** $10^9 \times 2{,}2 / (4 \times 10^9)$
**= 0,550 s**

**MIPS =** 4000 / 2,2 ≈ 1818

**Speedup B/A**

0,567 / 0,550

**≈ 1,03×**

con 33 % más GHz

**Lección:** los GHz por sí solos no miden el rendimiento. Importan IC, CPI y f juntos.

Laboratorio de la CPU · pestaña 2

# Taller · Ejercicio 2: Ley de Amdahl

**Ejercicio 2**

**Un programa de análisis de datos pasa el 80 % de su tiempo en una parte paralelizable.**

**a** ¿Qué speedup se obtiene con 8 núcleos?

**b** ¿Cuál es el speedup máximo posible, aunque haya infinitos núcleos?

**c** ¿Cuántos núcleos se necesitan para lograr un speedup de 4?

**Recuerda:** Speedup = 1 / [ (1 – f) + f / N ]

# Ejercicio 2 · Solución

**a**

**8 núcleos**

1 / (0,2 + 0,8/8) = 1 / 0,3

**3,33×**

**b**

**Máximo (N → ∞)**

1 / (1 – 0,8) = 1 / 0,2

**5×**

**c**

**Speedup = 4**

0,2 + 0,8/N = 0,25 → 0,8/N = 0,05

**N = 16**

**Rendimientos decrecientes:** duplicar los núcleos (8 → 16) solo lleva el speedup de 3,33× a 4×.

# Taller · Ejercicio 3: tiempo medio de acceso

**Ejercicio 3**

**Una caché tiene tiempo de acierto de 1 ns y la RAM una penalización por fallo de 100 ns.**

**a** Calcula el AMAT si la tasa de aciertos es 95 %.

**b** Calcula el AMAT si la tasa de aciertos sube a 99 %.

**c** ¿Cuántas veces más rápido es el acceso medio con 99 % frente a no tener caché (100 ns)?

**Recuerda:** AMAT = $t_{hit}$ + (1 – h) × penalización

# Ejercicio 3 · Solución

**a**

**h = 95 %**

1 + 0,05 × 100

**= 6 ns**

**b**

**h = 99 %**

1 + 0,01 × 100

**= 2 ns**

**c**

**Frente a sin caché**

100 ns / 2 ns

**= 50×**

**De 95 % a 99 % de aciertos:** el acceso medio es 3 veces más rápido.

**Laboratorio de la CPU** · pestaña 3: AMAT

# Quiz de la semana · Pregunta 1 de 6

**¿Qué registro contiene la dirección de la PRÓXIMA instrucción a ejecutar?**

**A)** IR

**B)** MAR

**C)** PC

**D)** MBR

# Quiz de la semana · Respuesta 1 de 6

**Pregunta:** ¿Qué registro contiene la dirección de la próxima instrucción?

**A)** IR

**B)** MAR

**C)** **PC** ✓

**D)** MBR

### ¿Por qué?

El **PC** (contador de programa) apunta a la próxima instrucción. En la búsqueda se copia al MAR y luego se incrementa.

**IR** tiene la instrucción actual; **MAR** la dirección del acceso en curso; **MBR** el dato.

# Quiz de la semana · Pregunta 2 de 6

**lscpu muestra «L1d cache» y «L1i cache» por separado. ¿Qué idea de arquitectura refleja?**

**A)** Von Neumann puro

**B)** Harvard modificada

**C)** Multiprocesamiento simétrico

**D)** Memoria virtual

# Quiz de la semana · Respuesta 2 de 6

**Pregunta:** L1d y L1i separadas reflejan...

A) Von Neumann puro

B) **Harvard modificada** ✓

C) Multiprocesamiento simétrico

D) Memoria virtual

**¿Por qué?**

Separar la caché L1 en instrucciones (**L1i**) y datos (**L1d**) permite captar ambos a la vez: es la idea de Harvard.

Pero la memoria principal es única (Von Neumann). Por eso se llama **Harvard modificada**.

# Quiz de la semana · Pregunta 3 de 6

**El Intel 8086 tenía un bus de direcciones de 20 bits. ¿Cuánta memoria podía direccionar?**

**A)** 64 KiB

**B)** 1 MiB

**C)** 20 MB

**D)** 4 GiB

# Quiz de la semana · Respuesta 3 de 6

**Pregunta:** Bus de direcciones de 20 bits → memoria direccionable

**A)** 64 KiB

**B)** **1 MiB** ✓

**C)** 20 MB

**D)** 4 GiB

**¿Por qué?**

$2^{20}$ = 1 048 576 direcciones de byte = **1 MiB**.

Regla: n bits de dirección → $2^n$ direcciones. 4 GiB corresponde a 32 bits; 64 KiB, a 16 bits.

# Quiz de la semana · Pregunta 4 de 6

**Hay que copiar un archivo de 2 GB del SSD a la memoria. ¿Qué técnica de E/S libera más a la CPU?**

**A)** E/S programada

**B)** E/S por interrupciones

**C)** DMA

**D)** Sondeo (polling)

# Quiz de la semana · Respuesta 4 de 6

**Pregunta:** Copiar 2 GB del SSD a memoria: ¿qué técnica?

**A)** E/S programada

**B)** E/S por interrupciones

**C)** **DMA** ✓

**D)** Sondeo (polling)

### ¿Por qué?

Con **DMA** el controlador mueve el bloque completo y solo interrumpe al final.

Con E/S programada o por interrupciones, la CPU movería cada palabra: millones de operaciones para 2 GB.

# Quiz de la semana · Pregunta 5 de 6

**Una CPU de 2 GHz ejecuta un programa de $4 \times 10^9$ instrucciones con CPI promedio de 1,5. ¿Cuál es el tiempo de CPU?**

**A)** 1,5 s

**B)** 3 s

**C)** 6 s

**D)** 12 s

# Quiz de la semana · Respuesta 5 de 6

**Pregunta:** f = 2 GHz, IC = $4 \times 10^9$, CPI = 1,5 → T = ?

**A)** 1,5 s

**B)** **3 s** ✓

**C)** 6 s

**D)** 12 s

**¿Por qué?**

**T = IC × CPI / f**

T = $4 \times 10^9 \times 1{,}5 / (2 \times 10^9) = 6 \times 10^9 / 2 \times 10^9$ = **3 s**

Error típico: olvidar dividir por f (6 s) o multiplicar por f (12 s).

# Quiz de la semana · Pregunta 6 de 6

**Solo el 50 % de un programa puede paralelizarse. ¿Cuál es el speedup máximo con infinitos núcleos?**

**A)** 1,5×

**B)** 2×

**C)** 50×

**D)** Infinito

# Quiz de la semana · Respuesta 6 de 6

**Pregunta:** f = 0,5 y núcleos infinitos → speedup máximo

A) 1,5×

B) **2×** ✓

C) 50×

D) Infinito

**¿Por qué?**

Límite de Amdahl: **1 / (1 – f) = 1 / 0,5 = 2**.

La mitad secuencial no se acelera con más núcleos: aunque la parte paralela tarde cero, el programa tarda al menos la mitad del tiempo original.

# Deber 1 · Script de inventario de hardware

### El reporte debe incluir

- Nombre del equipo, fecha y sistema operativo
- CPU: modelo, núcleos, hilos y frecuencia
- Tamaños de caché L1, L2 y L3
- Memoria RAM total
- Discos: nombre, tipo y tamaño
- Guardar la salida en inventario_<equipo>.txt

**Extra:** interfaces de red y dirección IP

**Modalidad**
En parejas (las mismas del portafolio)

**Entrega**
GitHub · carpeta semana01/ · enlace por el aula virtual

**Fecha**
[completar]

**Rúbrica (25 % c/u)**
Funciona en ambos SO · documentado · organizado · commits de ambos

# Deber 1 · Esqueleto para empezar

inventario.sh

```
#!/usr/bin/env bash
# inventario.sh · Semana 1
salida="inventario_$(hostname).txt"
{
  echo "== Equipo: $(hostname) | $(date)"
  echo "== SO: $(. /etc/os-release; echo $PRETTY_NAME)"
  echo "== CPU"
  lscpu | grep -E 'Model name|^CPU\(s\)|Thread|L1|L2|L3'
  echo "== Memoria"; free -h | head -2
  echo "== Discos";  lsblk -d -o NAME,SIZE,TYPE
  # Pendiente: modelo del disco y extra de red
} > "$salida"
echo "Reporte generado: $salida"
```

inventario.ps1

```
# inventario.ps1 · Semana 1
$salida = "inventario_$env:COMPUTERNAME.txt"
$cpu = Get-CimInstance Win32_Processor
$os  = Get-CimInstance Win32_OperatingSystem
& {
  "== Equipo: $env:COMPUTERNAME | $(Get-Date)"
  "== SO: $($os.Caption) $($os.Version)"
  "== CPU"
  $cpu | Format-List Name, NumberOfCores,
                     NumberOfLogicalProcessors
  "== RAM (GB): {0:N1}" -f ($os.TotalVisibleMemorySize/1MB)
  "== Discos"
  Get-PhysicalDisk | Format-Table FriendlyName, Size
  # Pendiente: frecuencia, cachés L2/L3 y red
} | Out-File $salida
Write-Host "Reporte generado: $salida"
```

# Conclusiones de la semana

## Bloques

CPU, memoria, E/S y buses, organizados según Von Neumann (con cachés L1 Harvard). Se verifican con comandos en Linux y Windows.

## Funcionamiento

Ciclo de búsqueda y ejecución guiado por el PC; las interrupciones y el DMA permiten solapar CPU y E/S.

## Rendimiento

T = IC × CPI / f. Los GHz solos no bastan; Amdahl limita lo que aporta mejorar una parte; la caché reduce el AMAT.

**Semana 2:** 1.4 Modelo de programación · 1.5 Jerarquía de memoria · 1.6 Representación de datos.

# Bibliografía y recursos de interés

## Bibliografía


- Stallings, W. (2006). *Organización y arquitectura de computadores* (7.ª ed.). Pearson Prentice Hall. Caps. 1–3 y 7.
- Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson. Cap. 1.
- Carretero, J., García, F., de Miguel, P. y Pérez, F. (2007). *Sistemas operativos: una visión aplicada* (2.ª ed.). McGraw-Hill. Cap. 1.
- Tanenbaum, A. S. y Bos, H. (2022). *Modern Operating Systems* (5th ed.). Pearson. Sec. 1.3.


## Recursos en línea

- **Laboratorio de la CPU (del curso)**
- Little Man Computer (simulador de CPU) peterhigginson.co.uk/lmc/
- CPUlator (simulador de ensamblador) cpulator.01xz.net/
- Visual 6502 (CPU real simulada) www.visual6502.org/
- Compiler Explorer godbolt.org/
- Latencias interactivas colin-scott.github.io/personal_websi...
- OSTEP (libro libre) pages.cs.wisc.edu/~remzi/OSTEP/